

# Eco-Resilient Networks: Smart Deployment for the Future\*

1<sup>st</sup> Muhammad Hamza Shahab  
Faculty of Engineering and Applied Science  
University of Regina  
Regina, Canada  
msx044@uregina.ca

2<sup>nd</sup> Syed Haider Abbas Nqavi  
Faculty of Engineering and Applied Science  
University of Regina  
Regina, Canada  
snh436@uregina.ca

**Abstract**—The rapid evolution of network services, driven by innovations like virtualization and edge computing, has transformed the way modern applications are deployed and managed. Service Function Chains (SFC) formed by putting Virtual Network Functions (VNFs) in a particular order enable flexible and scalable network solutions. However, their virtualized nature introduces new challenges, including their availability and sustainability. Their joint balancing in SFCs deployment is crucial to meet the stringent requirements of next-generation networks. This study presents a novel approach for availability and sustainability-aware SFCs allocation and embedding in the edge-cloud continuum. It introduces embedding policies tailored to prioritize availability, reduce carbon footprint, or achieve a tradeoff between the two. Simulated Annealing (SA), Particle Swarm Optimization (PSO) and Genetic Algorithms (GA) are employed to optimize VNF redundancy strategies. Simulation results demonstrate the effectiveness of the proposed methods, achieving robust fault tolerance while minimizing carbon footprint. The tradeoff-aware policy and PSO based redundancy strategy achieves 95.88% availability while cutting the carbon footprint by 37.6%. These strategies align with UN Sustainable Development Goals (SDGs) 7, 11, 12, and 13, by promoting affordable and clean energy, building resilient infrastructure, fostering sustainable cities, and encouraging responsible consumption and production.

**Index Terms**—Service Function Chains, Availability, Carbon Footprint, Latency, Edge-cloud continuum, Multi-objective optimization

## I. INTRODUCTION

The emergence of Network Function Virtualization (NFV) and Software-Defined Networking (SDN) has revolutionized network service provisioning, offering increased flexibility, scalability, and cost-effectiveness. By decoupling network functions from dedicated hardware, NFV enables the creation of Service Function Chains (SFCs), which are ordered sequences of Virtual Network Functions (VNFs) that provide specific services [1]. Edge computing extends this paradigm by bringing computation and storage resources closer to end-users, reducing latency and improving the performance of delay-sensitive applications. However, the dynamic and distributed nature of edge-cloud environments introduces new challenges in terms of ensuring both service availability and environmental sustainability. This research is driven by the need to address the increasing demands on network infrastructure while minimizing environmental impact, aligning with

the global focus on sustainable development. The project's community characteristics orientation centers on SaskTel's network engineers and architects, emphasizing the importance of user-centric design and the integration of new tools with existing workflows. The business case opportunity lies in providing SaskTel with solutions that enhance network availability and performance, reduce operational costs, and support their commitment to corporate social responsibility. A thorough stakeholder analysis has been conducted to identify key stakeholders, including SaskTel, end-users, and other telecommunications companies, and to understand their needs and expectations. Traditional network design approaches often prioritize either availability or sustainability, leading to trade-offs. For instance, increasing redundancy to enhance availability can lead to higher energy consumption and carbon emissions. There is a need for novel approaches that can jointly optimize these conflicting objectives. This paper addresses this challenge by proposing a set of SFC allocation and embedding policies that consider both availability and sustainability in edge-cloud environments. The main contributions of this paper are:

- Proposes a comprehensive approach for availability and sustainability aware SFC allocation and embedding in the edge-cloud continuum.
- Introduces novel algorithms that optimize placement and redundancy with joint consideration of availability and sustainability under resource constraints.
- Evaluates the proposed methods using realistic datasets and benchmarks to highlight the effectiveness of proposed methods in achieving sustainable and reliable network services.

The paper is structured as follows: Section II discuss the related research works. Section III defines the problem statement. Section IV outlines the proposed approach and algorithms. Section V presents results and analysis. Finally, Section VI concludes the work and discusses the future directions.

## II. RELATED WORKS

In this section, we review the most closely related works to our research. A comprehensive systematic survey on this topic is available in [2]. Our analysis reveals that existing research on SFC embedding and orchestration has primarily addressed

the availability, and sustainability as separate concerns. Fan et al. proposed an availability-aware SFC mapping algorithm that minimizes resource consumption while ensuring availability but does not consider energy optimization [3]. Khosravi et al. introduced energy-efficient VM placement approaches to reduce energy and carbon costs, but they overlook VNF-specific availability and edge-cloud challenges [4]. Araujo et al. suggested shared-path backup strategies to improve resource efficiency, but their method primarily focuses on ensuring availability and does not consider energy-related metrics [5]. Mai et al. developed an energy-efficient VNF placement policy with service availability guarantees but focused on static scenarios, ignoring edge-cloud dynamics [6]. Wang et al. explored parallelized SFC placement in data centers, optimizing latency and traffic, but neglected energy impacts [7]. To address these limitations, our work proposes a holistic approach to enhance the availability and sustainability of SFCs ensuring high availability while minimizing the carbon footprint.

### III. PROBLEM STATEMENT AND FORMULATION

The objective is to optimize the number of backup instances of VNFs and place them in an edge-cloud continuum in such a way that it, balances the tradeoff between availability and carbon footprint of the system. The notations used in the following sections are listed in Table I.

#### A. System model

We model an edge-cloud continuum with nodes arranged in a fat-tree topology [8]. The substrate network is an undirected graph  $G = (N, L)$ , where  $N$  represents the heterogeneous edge-cloud nodes with CPU, memory, and storage resources, and  $L$  represents the physical links connecting them. Service chain requests ( $R$ ) are modeled as directed graphs  $r = (F, V)$ , where  $F$  is the set of VNFs and  $V$  is the set of virtual links. Each VNF  $f \in F$  has specific resource demands, and each link  $v \in V$  requires a specified bandwidth. VNFs are hosted on nodes that satisfy resource constraints, and virtual links are mapped to physical paths made of physical links  $L$  between hosting nodes that meet bandwidth and delay constraints. Binary variables  $X_f^n$  and  $Y_v^l$  indicate whether a VNF  $f$  is placed on node  $n$  ( $X_f^n = 1$ ) and whether a virtual link  $v$  is mapped to a physical link  $l$  ( $Y_v^l = 1$ ). The following sections detail the availability and carbon footprint models, used to quantify the impact of SFC embedding and redundancy.

#### B. Availability model

To model the system availability, we consider the availabilities of VNFs, nodes, and network switches, assuming 100% availability for both physical and virtual links. The availability of each component is calculated using its MTBF and MTTR (Eq. 1). Series configurations represent sequential dependencies, such as VNFs in an SFC or switches along a path, where a single failure disrupts the chain (Eq. 2). Parallel configurations capture redundancies, like multiple paths or redundant VNFs, ensuring functionality despite failures (Eq. 3). For a VNF

placed on a node, availability is the product of their individual availabilities (Eq. 4). Path availability considers end nodes and intermediate switch availabilities (Eq. 5), while overall system availability combines series and parallel components (Eq. 6) [7].

$$A = \frac{MTBF}{MTBF + MTTR} \quad (1)$$

$$A_{ser} = \prod_{i \in N} A_i \quad (2)$$

$$A_{para} = \prod_{i \in N} 1 - (1 - A_i) \quad (3)$$

$$A_f^n = A_f \cdot A_n \quad (4)$$

$$A_{path_{ij}} = A_{n_i} \cdot A_{n_j} \cdot \prod_{i \in S} A_i \quad (5)$$

$$A_{sys} = \prod_{i \in ser} A_i \cdot \prod_{j \in para} A_j \quad (6)$$

#### C. Carbon footprint model

The equations for calculating the energy consumption of nodes and switches, as well as the total carbon footprint of the system, are represented by Eq. (7) to (9) [9].

$$E_n = \begin{cases} [p_{min}^n + (p_{max}^n - p_{min}^n) \cdot u_{cpu}^n] \cdot t, & u_{cpu}^n > 0; \\ 0, & u_{cpu}^n = 0. \end{cases} \quad (7)$$

$$E_s = \begin{cases} (p_{static}^s + p_{port}^s \cdot q_s) \cdot t, & \text{switch } s \text{ is on;} \\ 0, & \text{switch } s \text{ is off.} \end{cases} \quad (8)$$

$$CF_{sys} = \left( \sum_{n \in N} E_n + \sum_{s \in S} E_s \right) \cdot \gamma_{CO2} \quad (9)$$

#### D. Objective Function

The objective is to balance the tradeoff between availability and carbon footprint, represented by Eq. 10, where  $\alpha$  determines the relative importance of each metric.

$$F(A_{sys}, CF_{sys}) = \max[\alpha A_{sys} - (1 - \alpha) CF_{sys}] \quad (10)$$

The equations for resource utilization and bandwidth constraints are represented by Eq. (11) to (14). The latency constraint ensuring maximum end-to-end delay is represented by Eq. 15.

$$\sum_{f \in F} (d_{cpu}^f X_f^n) \leq C_{cpu}^n, \forall n \in N \quad (11)$$

$$\sum_{f \in F} (d_{mem}^f X_f^n) \leq C_{mem}^n, \forall n \in N \quad (12)$$

$$\sum_{f \in F} (d_{st}^f X_f^n) \leq C_{st}^n, \forall n \in N \quad (13)$$

$$\sum_{v \in V} (d_{bw}^v Y_v^l) \leq C_{bw}^l, \forall l \in L \quad (14)$$

$$D_{end}^r < D_{max}^r, \forall r \in R \quad (15)$$

TABLE I: Notations used in this paper

| Symbol                             | Description                                     | Symbol                     | Description                                      |
|------------------------------------|-------------------------------------------------|----------------------------|--------------------------------------------------|
| $G, R$                             | Physical network, Set of all SFC requests       | $N, L, S$                  | Set of nodes, physical links, switches           |
| $F, V$                             | VNFs, virtual links                             | $X_f^n, Y_v^l$             | 1 if $f/v$ mapped to $n/l$ , 0 otherwise         |
| $\overline{MTBF}, \overline{MTTR}$ | Mean time between failure, to repair            | $A_{ser}^n, A_{para}^n$    | Serial, parallel component availability          |
| $A_n, A_f, A_s$                    | Availability of node $n$ , VNF $f$ , switch $s$ | $A_f^n$                    | Availability of VNF $f$ hosted on node $n$       |
| $A_{path_{ij}}$                    | Path availability between $i$ and $j$           | $A_{sys}$                  | System availability                              |
| $p_{min}^n, p_{max}^n$             | Min, max power consumption of node $n$          | $p_s^{static}, p_s^{port}$ | Static, per-port power consumption of switch $s$ |
| $u_{cpu}^n$                        | CPU utilization of node $n$                     | $q_s$                      | Active ports on switch $s$                       |
| $E_n, E_s$                         | Energy consumption of node $n$ , switch $s$     | $CF_{sys}$                 | Carbon footprint of system                       |
| $\gamma_{CO2}$                     | Carbon footprint rate                           | $\alpha$                   | Weight factor $\alpha \in [0, 1]$                |
| $C_{cpu}^n, C_{mem}^n, C_{st}^n$   | CPU, memory, storage capacity of node $n$       | $C_{bw}^l$                 | Bandwidth capacity of physical link $l$          |
| $d_{cpu}^f, d_{mem}^f, d_{st}^f$   | CPU, memory, storage demand of VNF $f$          | $d_{bw}^v$                 | Bandwidth demand of virtual link $v$             |
| $D_{end}^r$                        | End-end delay of SFC $r$                        | $D_{max}^r$                | Max tolerable delay of SFC $r$                   |

#### IV. AVAILABILITY AND SUSTAINABILITY AWARE APPROACH FOR SFCs

Figure 1 shows the approach to enhance the availability and sustainability of SFCs. It consists of two interconnected parts: Part 1 focuses on embedding SFCs. Part 2 optimizes the number of redundant instances to improve fault tolerance and minimize the carbon footprint. The process starts with Part 1, where primary VNFs are placed using availability-aware, carbon-aware, or tradeoff-aware policies. These placements are then passed to Part 2, which determines redundancy levels for each VNF using techniques like Simulated Annealing (SA), Particle Swarm Optimization (PSO), or Genetic Algorithm (GA). The redundant VNFs are embedded back using policies in Part 1, creating a closed-loop process driven by the objective function (Eq. 10). The loop continues until an optimal redundancy configuration is achieved or a pre-defined iteration limit is reached.

##### A. SFC embedding

This section discusses Part 1 of the approach in Figure 1. It focuses on SFC embedding (Algorithm 1) using following policies in the Policy-Specific Node Selection function:

- **Availability Aware Embedding (AAE):** This policy maximizes the availability by embedding SFCs on the most available nodes ( $\alpha = 1$ ).
- **Carbon Aware Embedding (CAE):** This policy minimizes the carbon footprint by embedding SFCs on the most energy-efficient nodes ( $\alpha = 0$ ).
- **Tradeoff Aware Embedding (TAE):** This policy balances availability and carbon footprint by considering both metrics in the objective function (Eq. 10), ensuring an optimized tradeoff in embedding decisions.

##### Algorithm 1 SFC Embedding

**Require:** Network  $G = (N, L)$ , SFCs  $R$ , Policy  $P$

**Ensure:** SFC embedding and link mapping

```

1:  $N' \leftarrow \text{SelectNodes}(N, P)$   $\triangleright$  Select nodes by policy
2: for each SFC  $r \in R$  do  $\triangleright$  For all SFCs
3:   for each VNF  $f_i \in r$  do  $\triangleright$  For VNFs in  $r$ 
4:      $placed \leftarrow \text{False}$   $\triangleright$  Placement status
5:     while  $N' \neq \emptyset$  and not  $placed$  do
6:        $n \leftarrow N'.pop(0)$   $\triangleright$  Get next node
7:       if  $\text{CheckConstraints}(n, f_i)$  then
8:          $placed \leftarrow \text{True}$   $\triangleright$  Mark as placed
9:       end if
10:    end while
11:    if not  $placed$  then return False
12:    end if
13:  end for
14: end for
15: return True

```

##### Function Policy-Specific Node Selection

```

1: procedure SELECTNODES( $N, P$ )
2:   if  $P = \text{AAE}$  then  $\triangleright$  Availability-aware policy
3:      $\alpha \leftarrow 1$   $\triangleright$  Weight for max. availability
4:      $score(n) \leftarrow A_n, \forall n \in N$ 
5:   else if  $P = \text{CAE}$  then  $\triangleright$  Carbon-aware policy
6:      $\alpha \leftarrow 0$   $\triangleright$  Weight for min. carbon footprint
7:      $score(n) \leftarrow -CF_n, \forall n \in N$ 
8:   else if  $P = \text{TAE}$  then  $\triangleright$  Tradeoff policy
9:      $\alpha \in (0, 1)$   $\triangleright$  Weight for tradeoff policy
10:     $score(n) \leftarrow \alpha A_n - (1 - \alpha) CF_n, \forall n \in N$ 
11:   end if
12:   Sort  $N$  by  $score$  (descending)  $\triangleright$  Sort nodes
13:   return  $N$   $\triangleright$  Return sorted list
14: end procedure

```

##### B. Optimal VNF redundancy configuration

This section explains Part 2 of the approach in Figure 1. It focuses on calculating the optimal number of redundant VNFs to enhance the availability and sustainability of SFCs using the following heuristic algorithms:

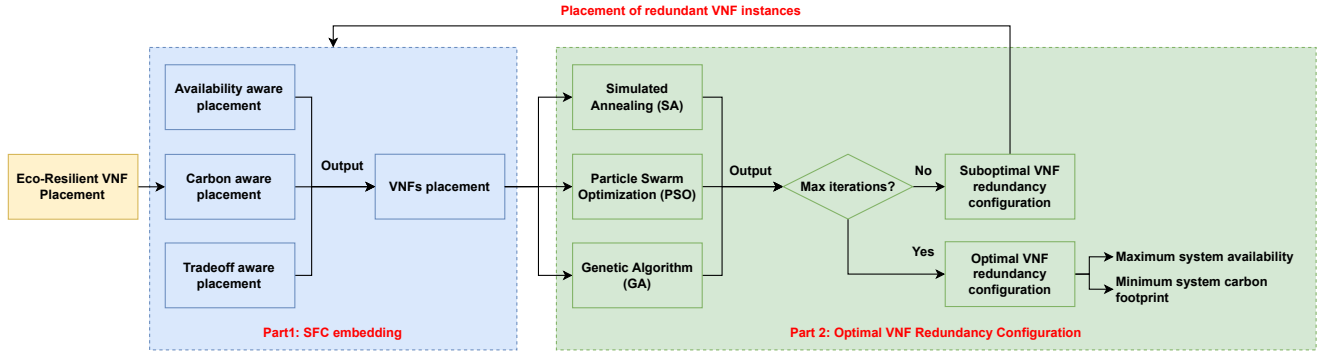


Fig. 1: A comprehensive approach to enhance the availability and sustainability in SFCs

- **Simulated Annealing (SA):** A probabilistic optimization method that adjusts redundancy configurations iteratively, avoiding local optima using a temperature-guided acceptance mechanism (Algorithm 2).
- **Particle Swarm Optimization (PSO):** An optimization algorithm inspired by swarm behavior, where particles refine redundancy levels by balancing exploration and local optimization (Algorithm 3).
- **Genetic Algorithm (GA):** An evolutionary algorithm that evolves redundancy configurations through selection, crossover, and mutation to optimize availability and resource usage (Algorithm 4).

Function Fitness Evaluator evaluates the fitness of redundancy configurations from SA, PSO, and GA by placing redundant VNFs on nodes and calculating their fitness by using Eq. 10.

#### Algorithm 2 Simulated Annealing (SA)

**Require:** Network  $G = (N, L)$ , SFCs  $R$

**Ensure:** Redundancy configuration  $x_{best}$

```

1:  $x \leftarrow \text{InitializeRedundancy}(R)$   $\triangleright$  Initial solution
2:  $T \leftarrow \text{InitialTemperature}$   $\triangleright$  Set starting temperature
3: while not TerminationCondition( $T$ ) do
4:    $x_{new} \leftarrow \text{GenerateNeighbor}(x)$ 
5:    $\Delta f \leftarrow \text{Fitness}(x_{new}, R) - \text{Fitness}(x, R)$ 
6:   if  $\Delta f > 0$  or Random()  $\leq e^{\Delta f/T}$  then
7:      $x \leftarrow x_{new}$   $\triangleright$  Update current solution
8:   end if
9:    $T \leftarrow \text{CoolingSchedule}(T)$   $\triangleright$  Cool temperature
10:  if  $\text{Fitness}(x, R) > \text{Fitness}(x_{best}, R)$  then
11:     $x_{best} \leftarrow x$   $\triangleright$  Update best solution
12:  end if
13: end while
14: return  $x_{best}$   $\triangleright$  Return best solution

```

### V. PERFORMANCE EVALUATION

This section evaluates the performance of the methods proposed in Sections IV-A and IV-B.

#### A. System Setup

The dataset used in this study is derived from the Google Cluster Traces 2019 [11]. These traces include 90,884 nodes

#### Algorithm 3 Particle Swarm Optimization (PSO)

**Require:** Network  $G = (N, L)$ , SFCs  $R$

**Ensure:** Redundancy configuration  $x_{gbest}$

```

1:  $P \leftarrow \text{InitializePopulation}(R)$   $\triangleright$  Initial particles
2:  $V \leftarrow \text{InitializeVelocities}()$   $\triangleright$  Set initial velocities
3:  $P_{best} \leftarrow P$   $\triangleright$  Store best position for particles
4:  $x_{gbest} \leftarrow \text{BestSolution}(P)$   $\triangleright$  Set global best solution
5: while not TerminationCondition() do
6:   for each particle  $p \in P$  do
7:      $v \leftarrow \text{UpdateVelocity}(p, v, P_{best}, x_{gbest})$ 
8:      $p \leftarrow \text{UpdatePosition}(p, v)$ 
9:     if  $\text{Fitness}(p, R) > \text{Fitness}(P_{best}[p], R)$  then
10:       $P_{best}[p] \leftarrow p$   $\triangleright$  Update personal best
11:    end if
12:   end for
13:    $x_{gbest} \leftarrow \text{BestSolution}(P)$   $\triangleright$  Update global best
14: end while
15: return  $x_{gbest}$   $\triangleright$  Return global best solution

```

#### Algorithm 4 Genetic Algorithm (GA)

**Require:** Network  $G = (N, L)$ , SFCs  $R$

**Ensure:** Redundancy configuration  $x_{best}$

```

1:  $Pop \leftarrow \text{InitializePopulation}(R)$   $\triangleright$  Initial solutions
2: while not TerminationCondition() do
3:    $F \leftarrow \text{Fitness}(Pop, R)$ 
4:    $Parents \leftarrow \text{Selection}(Pop, F)$   $\triangleright$  Select parents
5:    $Offspring \leftarrow \text{Crossover}(Parents)$ 
6:    $Offspring \leftarrow \text{Mutation}(Offspring)$ 
7:    $Pop \leftarrow \text{Replace}(Pop, Offspring)$ 
8: end while
9:  $x_{best} \leftarrow \text{BestSolution}(Pop)$   $\triangleright$  Update best solution
10: return  $x_{best}$   $\triangleright$  Return best solution

```

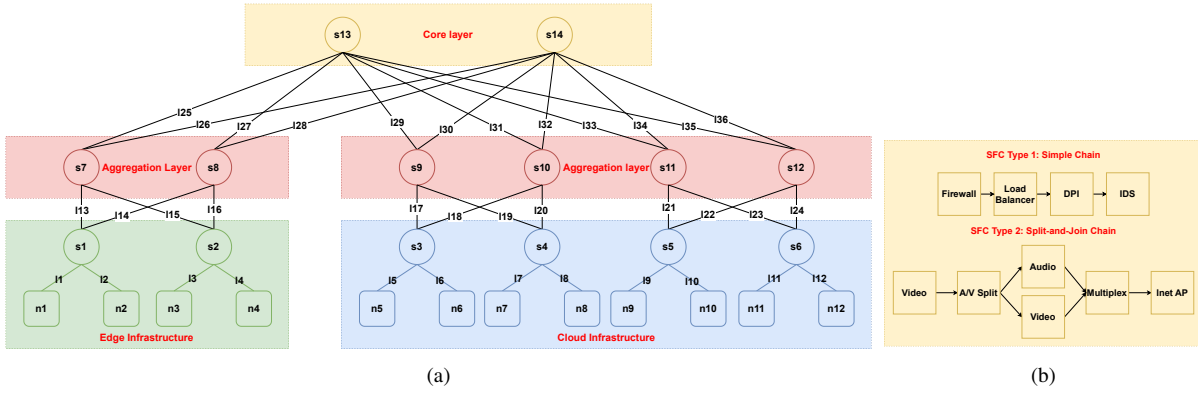


Fig. 2: An example of a substrate and virtual network considered in this paper (a) Fat Tree Topology (b) Service Function Chains

TABLE II: Configuration parameters for simulation

| Component    | Parameter              | Value                                                            |
|--------------|------------------------|------------------------------------------------------------------|
| Data Centers | Carbon Emission Rate   | 0.124-0.678 Tons/MWh [10]                                        |
|              | Number of nodes        | 90,884 [11]                                                      |
|              | Availability           | 90%-99.99% [11]                                                  |
| Nodes        | Resources              | CPU: 8-32 cores,<br>Memory: 2-32 GB,<br>Storage: 32-256 GB [12]  |
|              | Power (W)              | Idle: 52-161, Max: 144-268 [12]                                  |
|              | Availability           | Core: 99.9999%, Agg.: 99.99%, Edge: 99% [13]                     |
| Switches     | Static Power (W)       | Core: 300, Agg.: 150, Edge: 150 [14]                             |
|              | Power per port (W)     | 0.0005 [14]                                                      |
|              | Latency (ms)           | Core: 100, Agg.: 1, Edge: 1 [15]                                 |
|              | Instances              | 5, 10, 20                                                        |
| SFC          | SLA Availability       | 99.99%                                                           |
|              | Latency Threshold (ms) | 50-100 [16]                                                      |
|              | VNFs per SFC           | 4-6                                                              |
|              | Max Redundant VNFs     | 4 [17]                                                           |
|              | Availability           | 99.99% [17]                                                      |
| VNF          | Resource Demands       | CPU: 1-8 cores,<br>Memory: 100-400 MB,<br>Storage: 10-50 GB [16] |

TABLE III: Algorithm parameters for simulation

| Algorithm | Parameter                  | Value |
|-----------|----------------------------|-------|
| SA        | Initial Temperature        | 100   |
|           | Cooling Rate               | 0.95  |
|           | Iterations per Temperature | 100   |
|           | Stopping Temperature       | 0.1   |
| PSO       | Inertia Weight             | 0.7   |
|           | Cognitive Coefficient      | 1.5   |
|           | Social Coefficient         | 1.5   |
| GA        | Population Size            | 100   |
|           | Selection Probability      | 0.2   |
|           | Crossover Probability      | 0.9   |
|           | Mutation Probability       | 0.1   |

#### Function Fitness Evaluator

```

1: procedure FITNESS( $X, R$ ) ▷ Evaluate solutions
2:    $F \leftarrow \text{InitializeFitness}(X)$  ▷ Set initial fitness
3:   for each solution  $x \in X$  do ▷ Iterate solutions
4:     for each SFC  $r \in R$  do
5:       for each primary VNF  $f \in r$  do
6:          $i \leftarrow \text{RedundantInstances}(x, f)$ 
7:         if  $i > 0$  then PlaceVNF( $f, i, N, r$ )
8:         end if
9:       end for
10:    end for
11:     $F[x] \leftarrow \text{ObjectiveFunction}(N, R)$ 
12:  end for
13:  return  $F$  ▷ Return fitness values
14: end procedure

```

distributed across several clusters. Each node is associated with machine attributes, events, and resource capacities. We

calculate availability of each node by analyzing the corresponding time between failures and time to return events. Node energy consumption is estimated using the SpecPower2008 benchmark, with power profiles matched to the proportional CPU and memory capacities of nodes in the traces [12], [18]. Carbon emission rates are based on studies considering geographical variations in carbon intensity and Power Usage Effectiveness (PUE) [10]. The simulations use a fat-tree topology for edge and cloud data centers, as shown in Figure 2(a) [8]. Nodes are assigned to edge and cloud infrastructures based on their CPU capacities. Nodes having 16 or greater CPU cores are allocated to the cloud, while those having less than 16 cores are allocated to the edge. This results in 65,545 nodes (72.12%) in the cloud, characterized by higher resource capacities. In contrast, the edge comprises of 25,338 nodes (27.88%) with limited resources. We use two types of SFCs: (i) Simple Chains (e.g., web service through a firewall) (ii) Split-and-Join Chains (e.g., video streaming through a transcoder) as shown in Figure 2(b) [19]. VNFs are placed on nodes,

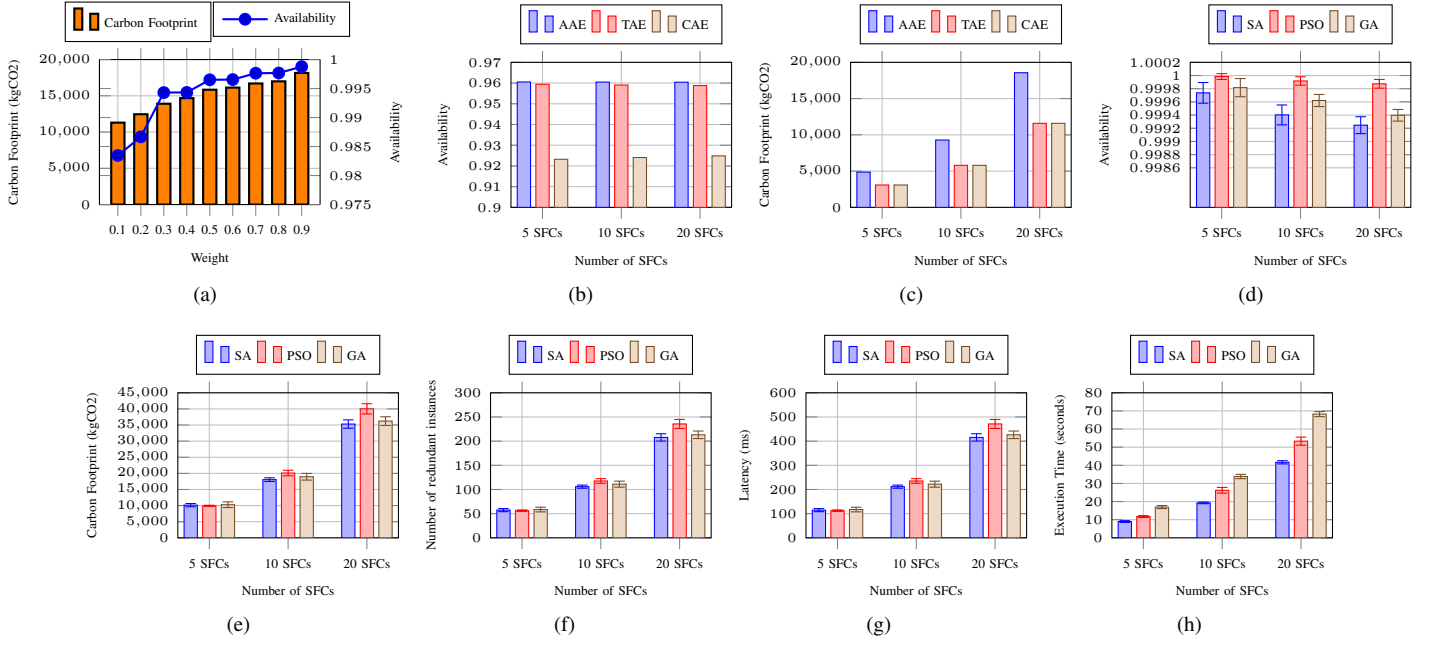


Fig. 3: Performance evaluation results of proposed algorithms (a) Availability and carbon footprint tradeoff w.r.t  $\alpha$ ; Placement policies: (b) Availability, (c) Carbon footprint; Redundancy policies: (d) Availability, (e) Carbon footprint, (f) Redundant instances, (g) Latency, (h) Execution time

and virtual links are mapped to the physical links using the shortest simple paths algorithm from the Python NetworkX library [20]. Simulations are run on a machine with an 8-core AMD Ryzen 7 CPU (3.2 GHz) and 16 GB RAM. Table II summarizes the configuration parameters for nodes, switches, SFCs, and VNFs, derived from existing literature to represent realistic values for edge and cloud environments. Table III presents the algorithm sepecific parameters.

### B. Simulation results

This section presents the performance evaluation results for the embedding and redundancy algorithms, along with key observations. As performance evaluation metrics, we used availability, carbon footprint (kgCO<sub>2</sub>), number of redundant instances, latency (ms) and execution times (seconds) achieved by the proposed policies and heuristic algorithms. Each simulation is executed 20 times, and results are reported with a 95% confidence level. Figure 3 illustrates the findings.

1) *Impact of weight on SFC embedding decision:* Figure 3(a) shows how the weight factor ( $\alpha$ ) affects availability and carbon footprint for 20 SFC requests. The weight factor controls the tradeoff between these metrics, enabling stakeholders to prioritize availability, sustainability, or a balance of both. As  $\alpha$  increases from 0.1 to 0.4, availability improves from 98.34% to 99.43%, a 1.1% increase, while the carbon footprint rises by 30%. Beyond  $\alpha = 0.5$ , availability increases at a slower rate, reaching 99.88% at  $\alpha = 0.9$ , marking a 1.57% total increase from the lowest weight value. Conversely, the carbon footprint continues to grow, reaching 18,156 kgCO<sub>2</sub> (a 60% increase compared to  $\alpha = 0.1$ ). Thus,  $\alpha = 0.4$  achieves

a good balance, offering a significant availability boost with a moderate increase in carbon footprint. This configuration is used in further simulations for tradeoff-aware SFC embedding policy.

2) *Performance of SFC embedding policies:* Figures 3(b) and (c) compare the performance of proposed SFC embedding policies in terms of availability and carbon footprint. The availability-aware policy achieves the highest availability by embedding the SFCs on the most available resources without considering their energy profiles. For example, for 20 SFCs, the availability-aware policy achieves 96.04% availability but results in a 18568.8 kgCO<sub>2</sub> carbon footprint. In contrast, the carbon-aware policy minimizes the carbon footprint by prioritizing the most energy-efficient nodes, at the cost of reduced availability. Compared to availability-aware, it reduces carbon footprint by 37.6% for 20 SFCs but lowers availability by 3.7%. The tradeoff-aware policy strikes a balance, delivering near-optimal availability with a significantly lower carbon footprint. For 20 SFCs, the availability is only 0.17% lower than availability-aware policy, while carbon footprint is reduced by 37.6%. These trends hold consistently across 5, 10, and 20 SFCs, demonstrating the scalability of the algorithms.

3) *Evaluation of VNF redundancy optimization heuristics:* Figures 3(d)-(h) presents the evaluation of heuristic algorithms for VNF redundancy. PSO achieves the highest availability, reaching 99.998% for 5 SFCs and 99.987% for 20 SFCs, outperforming SA and GA. While PSO incurs a 13% higher carbon footprint than SA, this tradeoff is justified by its superior fault tolerance, achieved through deploying the highest number of redundant VNFs (235.6 instances for 20 SFCs).

(Figures 3(d)-(f)). SA, on the other hand, minimizes the carbon footprint and achieves the lowest latency of 415.4 ms and execution time of 41.6 seconds for 20 SFCs but delivers slightly lower availability (99.925% for 20 SFCs) (Figures 3(d), (e), (g) & (h)). GA balances availability and carbon footprint but incurs the longest execution time of 68.22 seconds, which is 65% higher than SA and 28% higher than PSO for 20 SFCs, reflecting its computational complexity (Figures 3(d), (e) & (h)). Overall, PSO emerges as the best heuristic, delivering the highest availability and fault tolerance with acceptable increases in carbon footprint and latency, making it the most suitable choice for reliability and sustainability critical VNF redundancy optimization.

## VI. CONCLUSION AND FUTURE DIRECTIONS

This paper presented a novel approach for availability and sustainability-aware SFC allocation and embedding in edge-cloud environments. We introduced three embedding policies, AAE, CAE, and TAE, which can prioritize availability, reduce carbon footprint, or achieve a balance between the two. We also employed SA, PSO, and GA to optimize VNF redundancy strategies and evaluated their performance through simulations. The results demonstrate the effectiveness of the proposed methods in achieving robust fault tolerance while minimizing carbon footprint. The tradeoff-aware policy and PSO-based redundancy strategy offer a promising solution for balancing these two conflicting objectives. This research contributes to the advancement of sustainable network design, aligning with the UN Sustainable Development Goals (SDGs). The findings can guide the development of future network architectures and management strategies that are both resilient and environmentally responsible. Furthermore, the methodologies and tools developed in this study can be adapted and extended to address other sustainability challenges in various domains. Future work will focus on extending the framework to support dynamic service demands and network conditions, exploring more advanced optimization algorithms, and investigating the application of the proposed approach to real-world network deployments.

## ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to Professor Tim Maciag for their insightful feedback, guidance, and support throughout this project. This work was supported by the University of Regina, Faculty of Engineering and Applied Science.

## REFERENCES

- [1] M. Ghaznavi, N. Shahriar, R. Ahmed, and R. Boutaba, "Service function chaining simplified," *CoRR*, vol. abs/1601.00751, 2016.
- [2] M. H. Shahab and Y. Sharma, "Fault tolerance in service function chains: A taxonomy, survey and future directions," 2024.
- [3] J. Fan, C. Guan, Y. Zhao, and C. Qiao, "Availability-aware mapping of service function chains," in *IEEE INFOCOM 2017-IEEE Conference on Computer Communications*, pp. 1–9, IEEE, 2017.
- [4] A. Khosravi, L. L. Andrew, and R. Buyya, "Dynamic vm placement method for minimizing energy and carbon cost in geographically distributed cloud data centers," *IEEE Transactions on Sustainable Computing*, vol. 2, no. 2, pp. 183–196, 2017.
- [5] I. M. Araújo, C. Natalino, H. Chen, M. De Andrade, D. L. Cardoso, and P. Monti, "Availability-guaranteed service function chain provisioning with optional shared backups," in *2020 16th International Conference on the Design of Reliable Communication Networks DRCN 2020*, pp. 1–6, IEEE, 2020.
- [6] L. Mai, Y. Ding, X. Zhang, L. Fan, S. Yu, and Z. Xu, "Energy efficiency with service availability guarantee for network function virtualization," *Future Generation Computer Systems*, vol. 119, pp. 140–153, 2021.
- [7] M. Wang, B. Cheng, S. Wang, and J. Chen, "Availability-and traffic-aware placement of parallelized sfc in data center networks," *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 182–194, 2021.
- [8] J. Alqahtani and B. Hamdaoui, "Rethinking fat-tree topology design for cloud data centers," in *2018 IEEE Global Communications Conference (GLOBECOM)*, pp. 1–6, IEEE, 2018.
- [9] G. Sun, Y. Li, H. Yu, A. V. Vasilakos, X. Du, and M. Guizani, "Energy-efficient and traffic-aware service function chaining orchestration in multi-domain networks," *Future Generation Computer Systems*, vol. 91, pp. 347–360, 2019.
- [10] A. Khosravi, S. K. Garg, and R. Buyya, "Energy and carbon-efficient placement of virtual machines in distributed cloud data centers," in *Euro-Par 2013 Parallel Processing: 19th International Conference, Aachen, Germany, August 26-30, 2013. Proceedings 19*, pp. 317–328, Springer, 2013.
- [11] J. Wilkes, C. Reiss, N. Deng, M. E. Haque, and M. Tirmazi, "Google cluster-usage traces v3," technical report, Google, April 2020. Updated versions: May, July, August 2020.
- [12] S. P. E. C. (SPEC), "Specpower\_ssj2008 results," 2024.
- [13] G. Moualla, T. Turetti, and D. Saucez, "An availability-aware sfc placement algorithm for fat-tree data centers," in *2018 IEEE 7th International Conference on Cloud Networking (CloudNet)*, pp. 1–4, IEEE, 2018.
- [14] M. S. Yoon and A. E. Kamal, "Power minimization in fat-tree sdn datacenter operation," in *2015 IEEE Global Communications Conference (GLOBECOM)*, pp. 1–7, IEEE, 2015.
- [15] J. Son and R. Buyya, "Latency-aware virtualized network function provisioning for distributed edge clouds," *Journal of Systems and Software*, vol. 152, pp. 24–31, 2019.
- [16] M. A. Khoshkholghi, M. Gokan Khan, K. Alizadeh Noghani, J. Taheri, D. Bhamare, A. Kassler, Z. Xiang, S. Deng, and X. Yang, "Service function chain placement for joint cost and latency optimization," *Mobile Networks and Applications*, vol. 25, no. 6, pp. 2191–2205, 2020.
- [17] G. L. Santos, T. Lynn, J. Kelner, and P. T. Endo, "Availability-aware and energy-aware dynamic sfc placement using reinforcement learning," *The Journal of Supercomputing*, pp. 1–30, 2021.
- [18] P. Garraghan, I. S. Moreno, P. Townend, and J. Xu, "An analysis of failure-related energy waste in a large-scale cloud environment," *IEEE Transactions on Emerging topics in Computing*, vol. 2, no. 2, pp. 166–180, 2014.
- [19] A. Fischer, D. Bhamare, and A. Kassler, "On the construction of optimal embedding problems for delay-sensitive service function chains," in *2019 28th International Conference on Computer Communication and Networks (ICCCN)*, pp. 1–10, IEEE, 2019.
- [20] N. Developers, "Networkx: Shortest simple paths documentation," 2024.